

Lab 12 - Expected output reference

ClusterIPs, Pod IPs, the node InternalIP, the LoadBalancer's auto-allocated nodePort and all ages are runtime values and will differ. The pinned nodePort: 30080 is the only network number that should match literally.

bash verification/checks.sh - clean run

```
== Lab 12 verification: Service types on kind ==
PASS namespace kcna-lab12 exists
PASS deployment booking-edge has 2/2 ready replicas
PASS matrix row booking-edge-clusterip: type=ClusterIP port=80 targetPort=5678
PASS matrix row booking-edge-clusterip: clusterIP allocated as expected
PASS matrix row booking-edge-clusterip: no nodePort allocated, as expected
PASS matrix row booking-edge-nodeport: type=NodePort port=80 targetPort=5678
PASS matrix row booking-edge-nodeport: clusterIP allocated as expected
PASS matrix row booking-edge-nodeport: nodePort is 30080
PASS matrix row booking-edge-nodeport: nodePort 30080 is inside 30000-32767
PASS matrix row booking-edge-lb: type=LoadBalancer port=80 targetPort=5678
PASS matrix row booking-edge-lb: clusterIP allocated as expected
PASS matrix row booking-edge-lb: auto-allocated nodePort 31544 is inside 30000-32767
PASS EXPECTED ON kind: booking-edge-lb EXTERNAL-IP is still <pending> (no cloud-controller-manager)
PASS matrix row booking-edge-headless: type=ClusterIP port=80 targetPort=5678
PASS matrix row booking-edge-headless: clusterIP is None as expected
PASS matrix row booking-edge-headless: no nodePort allocated, as expected
PASS all four Services resolve to the same 2 endpoint addresses
PASS ClusterIP Service serves the exact text in data/edge-banner.txt
PASS NodePort 30080 answers on the node InternalIP from inside the cluster
PASS headless Service DNS returns Pod IPs, not a virtual IP
-- 20 passed, 0 failed --
```

Exit status 0.

The deliberately unusual check

```
PASS EXPECTED ON kind: booking-edge-lb EXTERNAL-IP is still <pending> (no cloud-controller-manager)
```

This check passes when the LoadBalancer has no external address. That is not a trick: on a stock kind cluster the absence of an address is the correct state, and data/service-matrix.csv records it as pending-forever. If you run this lab on EKS, GKE, AKS, or on kind with cloud-provider-kind/MetalLB installed, the check will correctly report:

```
FAIL booking-edge-lb unexpectedly has an external address '203.0.113.40' - this cluster is NOT stock kind, so revisit data/service-matrix.csv
```

That "failure" means your environment is *better* than the training cluster. Update the matrix row rather than the cluster.

Key intermediate outputs

All four Services together (Step 7)

NAME SELECTOR	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
booking-edge-clusterip app=booking-edge	ClusterIP	10.96.51.7	<none>	80/TCP	6m
booking-edge-headless app=booking-edge	ClusterIP	None	<none>	80/TCP	1m
booking-edge-lb app=booking-edge	LoadBalancer	10.96.184.221	<pending>	80:31544/TCP	3m
booking-edge-nodeport app=booking-edge	NodePort	10.96.77.140	<none>	80:30080/TCP	5m

Read the table as evidence of the nesting:

Row	ClusterIP?	nodePort?	External address?
booking-edge-clusterip	yes	no	no
booking-edge-nodeport	yes	yes (30080)	no
booking-edge-lb	yes	yes (31544, auto)	requested, never granted on kind
booking-edge-headless	None	no	no

LoadBalancer status is genuinely empty

```
$ kubectl get service booking-edge-lb -n kcna-lab12 -o jsonpath='{.status.loadBalancer}'
{}
```

```
$ kubectl describe service booking-edge-lb -n kcna-lab12 | tail -6
Session Affinity:      None
External Traffic Policy: Cluster
Internal Traffic Policy: Cluster
Events:                <none>
```

Events: <none> is the tell. A cluster with MetalLB would show IPAllocated / nodeAssigned; a cloud cluster would show EnsuringLoadBalancer / EnsuredLoadBalancer. An empty event list means no controller is watching the object at all.

Headless DNS versus ClusterIP DNS

ClusterIP - one virtual address:

```
Name: booking-edge-clusterip.kcna-lab12.svc.cluster.local
Address: 10.96.51.7
```

Headless - one record per ready Pod:

```
Name: booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.31
Name: booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.32
```

checks.sh queries the full FQDN here on purpose: BusyBox nslookup does not walk the /etc/resolv.conf search list once a name contains a dot, so only an absolute name gives a reliable answer from that tool.

Failure-injection reference (Section 6)

```
$ kubectl apply -f manifests/70-service-nodeport-invalid.yaml
The Service "booking-edge-nodeport-invalid" is invalid: spec.ports[0].nodePort: Invalid value:
8080: provided port is not in the valid range. The range of valid ports is 30000-32767
```

```
$ kubectl get service booking-edge-nodeport-invalid -n kcna-lab12
Error from server (NotFound): services "booking-edge-nodeport-invalid" not found
```

The same file passes static schema validation - run it yourself:

```
$ kubeconform -strict -summary manifests/70-service-nodeport-invalid.yaml
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

Schema-valid ≠ admissible. Port-range and allocation rules are enforced by the API server's validation and allocator, not by the OpenAPI schema.

Duplicate allocation:

```
$ kubectl create service nodeport clash --tcp=80:5678 --node-port=30080 -n kcna-lab12
The Service "clash" is invalid: spec.ports[0].nodePort: Invalid value: 30080: provided port is
already allocated
```

Environment caveats recorded for this lab

Caveat	Observed effect	Why
type: LoadBalancer never leaves <pending> on kind	EXTERNAL-IP column shows <pending> indefinitely; .status.loadBalancer is {}; Events: <none>	kind ships no cloud-controller-manager and no MetalLB. Nothing watches type=LoadBalancer Services. cloud-provider-kind or MetalLB would fulfil it
NodePort is not reachable from a macOS/Windows host	curl http://<nodeIP>:30080/ times out (exit 28) from the laptop, but succeeds from a Pod	Docker's bridge network lives inside a VM on those platforms. On native Linux Docker the same curl usually succeeds
extraPortMappings is create-time only	Cannot be added to a running kind cluster	Requires kind create cluster --config (Appendix A)
ClusterIP is unreachable from the host	curl http://10.96.x.y/ times out	The service CIDR is virtual; it exists only as kube-proxy rules inside cluster network namespaces. Use kubectl port-forward
nodePort is a cluster-wide allocation	A second Service claiming 30080 is rejected across all namespaces	The allocator is global, which is the practical argument for Ingress (Lab 13)
hashicorp/http-echo:1.0 must be pullable	ImagePullBackOff in an air-gapped room	Substitute nginx:1.27-alpine and set targetPort: 80 in all four Services; the Service-type behaviour under test is unchanged

